Informed Search in Python

Informed search algorithms

So far, we have talked about the uninformed search algorithms which looked through search space for all possible solutions of the problem without having any additional knowledge about search space. But informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node, etc. This knowledge help agents to explore less to the search space and find more efficiently the goal node.

The informed search algorithm is also called **heuristic search** or **directed search**. In contrast to uninformed search algorithms, informed search algorithms require details such as distance to reach the goal, steps to reach the goal, cost of the paths which makes this algorithm more efficient. Here, the goal state can be achieved by using the heuristic function.

The **heuristic function** is used to achieve the goal state with the lowest cost possible. This function estimates how close a state is to the goal. The heuristic function finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by h(n), and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

Admissibility of the heuristic function is given as:

1. h(n) <= h*(n)

Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues unit a goal state is found.

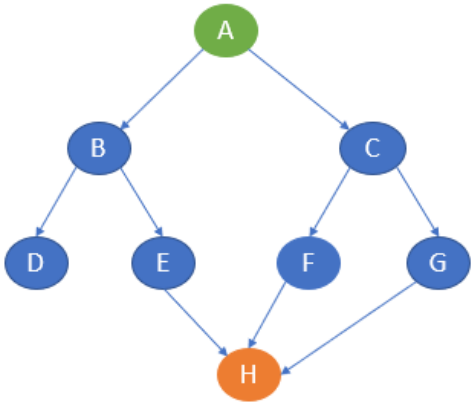
In the informed search we will discuss two main algorithms, Greedy Search and A* Search, which are given below:

- **Greedy Best First Search Algorithm (Greedy search)**
- **A* Search Algorithm**

1. Greedy best-first search algorithm

Greedy best-first search uses the properties of both depth-first search and breadth-first search. Greedy best-first search traverses the node by selecting the path which appears best at the moment. The closest path is selected by using the heuristic function.

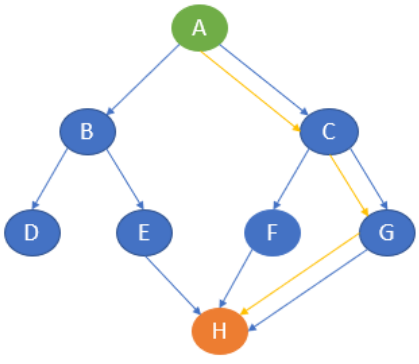
Consider the below graph with the heuristic values.



NODES	HEURISTICS
A	13
B	12
C	4
D	7
E	3
F	8
G	2
H	0

Here, A is the start node and H is the goal node.

Greedy best-first search first starts with A and then examines the next neighbour B and C. Here, the heuristics of B is 12 and C is 4. The best path at the moment is C and hence it goes to C. From C, it explores the neighbours F and G. the heuristics of F is 8 and G is 2. Hence it goes to G. From G, it goes to H whose heuristic is 0 which is also our goal state.



The path of traversal is A —-> C —-> G —-> H

Implementation in Python

The following is the implementation of the Greedy algorithm in Python.

```
graph = {
'A':[('B',12), ('C',4)],
'B':[('D',7), ('E',3)],
'C':[('F',8), ('G',2)],
'D':[],
'E':[('H',0)],
'F':[('H',0)],
'G':[('H',0)]
}
def bfs(start, target, graph, queue=[], visited=[]):
    if start not in visited:
        print(start)
        visited.append(start)
    queue=queue+[x for x in graph[start] if x[0][0] not in visited]
    queue.sort(key=lambda x:x[1])
    if queue[0][0]==target:
        print(queue[0][0])
    else:
        processing=queue[0]
        queue.remove(processing)
        bfs(processing[0], target, graph, queue, visited)
bfs('A', 'H', graph)
```

The output path with the lowest cost is generated.

```
A
C
G
H
```

The time complexity of Greedy best-first search is $O(b^m)$ in worst cases.

Advantages of Greedy best-first search

- Greedy best-first search is more efficient compared with breadth-first search and depth-first search.

Disadvantages of Greedy best-first search

- In the worst-case scenario, the greedy best-first search algorithm may behave like an unguided DFS.
- There are some possibilities for greedy best-first to get trapped in an infinite loop.
- The algorithm is not an optimal one.

Next, let's discuss the other informed search algorithm called the A* search algorithm.

---

2. A* search algorithm

A* search algorithm is a combination of both uniform cost search and greedy best-first search algorithms. It uses the advantages of both with better memory usage. It uses a heuristic function to find the shortest path. A* search algorithm uses the sum of both the cost and heuristic of the node to find the best path.

The A star (A*) algorithm is an algorithm used to solve the shortest path problem in a graph. This means that given a number of nodes and the edges between them as well as the "length" of the edges (referred to as "weight") and a heuristic (more on that later), the A* algorithm finds the shortest path from the specified start node to all other nodes. Nodes are sometimes referred to as vertices (plural of vertex) - here, we'll call them nodes.

Description of the Algorithm

The basic principle behind the A star (A*) algorithm is to iteratively look at the node with the currently smallest priority (which is the shortest distance from the start plus the heuristic to the goal) and update all not yet visited neighbours if the path to it via the current node is shorter. This is very similar to the Dijkstra algorithm, with the difference being that the lowest priority node is visited next, rather than the shortest distance node. In essence, Dijkstra uses the distance as the priority, whereas A* uses the distance plus the heuristic.

Why does adding the heuristic make sense? Without it, the algorithm has no idea if it's going in the right direction. When manually searching for the shortest path in this example, you probably prioritised paths going to the right over paths going up or down. This is because the goal node is to the right of the start node, so going right is at least generally the correct direction. The heuristic gives the algorithm this spatial information.

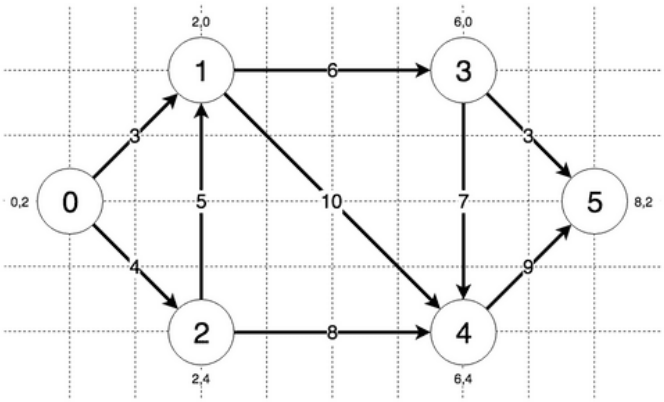
So, if a node has the currently shortest distance but is generally going in the wrong direction, whereas Dijkstra would have visited that node next, A Star will not. For this to work, the heuristic needs to be admissible, meaning it has to never overestimate the actual cost (i.e., distance) - which is the case for straight line distance in street networks, for example. Intuitively, that way the algorithm never overlooks a shorter path because the priority will always be lower than the real distance (if the current shortest path is A, then if there is any way path B could be shorter it will be explored). One simple heuristic that fulfils this property is straight line distance (e.g., in a street network)

In more detail, this leads to the following Steps:

1. Initialize the distance to the starting node as 0 and the distances to all other nodes as infinite
2. Initialize the priority to the starting node as the straight-line distance to the goal and the priorities of all other nodes as infinite
3. Set all nodes to “unvisited”
4. While we haven’t visited all nodes and haven’t found the goal node:
    1. Find the node with currently lowest priority (for the first pass, this will be the source node itself)
    2. If it’s the goal node, return its distance
    3. For all nodes next to it that we haven’t visited yet, check if the currently smallest distance to that neighbour is bigger than if we were to go via the current node
    4. If it is, update the smallest distance of that neighbour to be the distance from the source to the current node plus the distance from the current node to that neighbour, and update its priority to be the distance plus its straight-line distance to the goal node

Example of the Algorithm

Consider the following graph:



The steps the algorithm performs on this graph if given node 0 as a starting point and node 5 as the goal, in order, are:

1. Visiting node 0 with currently lowest priority of 8.0
2. Updating distance of node 1 to 3 and priority to 9.32455532033676
3. Updating distance of node 2 to 4 and priority to 10.32455532033676
4. Visiting node 1 with currently lowest priority of 9.32455532033676
5. Updating distance of node 3 to 9 and priority to 11.82842712474619
6. Updating distance of node 4 to 13 and priority to 15.82842712474619
7. Visiting node 2 with currently lowest priority of 10.32455532033676
8. Visiting node 3 with currently lowest priority of 11.82842712474619
9. Updating distance of node 5 to 12 and priority to 12.0
10. Goal node found!

Final lowest distance from node 0 to node 5: 12

Runtime of the Algorithm

The runtime complexity of A Star depends on how it is implemented. If a min-heap is used to determine the next node to visit, and adjacency is implemented using adjacency lists, the runtime is *O(|E| + |V|log|V|)* (*|V|* = number of Nodes, *|E|* = number of Edges). If we simply search all distances to find the node with the lowest distance in each step, and use a matrix to look up whether two nodes are adjacent, the runtime complexity increases to *O(|V|^2)*.

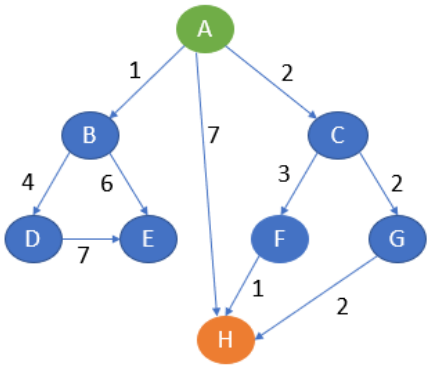
Note that this is the same as Dijkstra - in practice, though, if we choose a good heuristic, many of the paths can be eliminated before they are explored making for a significant time improvement.

More information on how the heuristic influences the complexity can be found on the Wikipedia Article.

Space of the Algorithm

The space complexity of A Star depends on how it is implemented as well and is equal to the runtime complexity, plus whatever space the heuristic requires.

Consider the following graph with the heuristics values as follows.



NODES	HEURISTICS
A	5
B	3
C	4
D	2
E	6
F	3
G	1
H	0

Let A be the start node and H be the goal node. First, the algorithm will start with A. From A, it can go to B, C, H. Note the point that A* search uses the sum of path cost and heuristics value to determine the path.

Here, from A to B, the sum of cost and heuristics is 1 + 3 = 4. From A to C, it is 2 + 4 = 6. From A to H, it is 7 + 0 = 7. Here, the lowest cost is 4 and the path A to B is chosen. The other paths will be on hold.

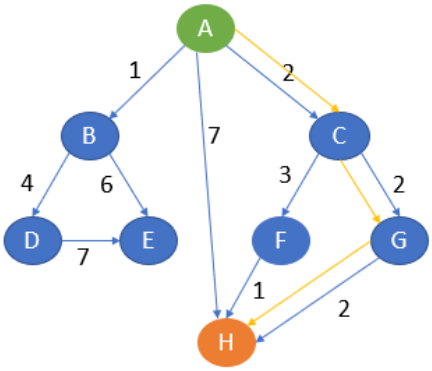
Now, from B, it can go to D or E. From A to B to D, the cost is 1 + 4 + 2 = 7. From A to B to E, it is 1 + 6 + 6 = 13. The lowest cost is 7. Path A to B to D is chosen and compared with other paths which are on hold.

Here, path A to C is of less cost. That is 6. Hence, A to C is chosen and other paths are kept on hold.

From C, it can now go to F or G. From A to C to F, the cost is 2 + 3 + 3 = 8. From A to C to G, the cost is 2 + 2 + 1 = 5. The lowest cost is 5 which is also lesser than other paths which are on hold. Hence, path A to G is chosen.

From G, it can go to H whose cost is 2 + 2 + 2 + 0 = 6. Here, 6 is lesser than other paths cost which is on hold.

Also, H is our goal state. The algorithm will terminate here.



The path of traversal is A —-> C —-> G —-> H

Implementation in Python

The following is the implementation of the A* algorithm in Python. First, I'm including a template (pseudocode) and then the concrete implementation in Python.

```
def a_star(graph, heuristic, start, goal):
    """

    Finds the shortest distance between two nodes using the A-star (A*) algorithm

    :param graph: an adjacency-matrix-representation of the graph where (x,y) is the
weight of the edge or 0 if there is no edge.

    :param heuristic: an estimation of distance from node x to y that is guaranteed to
be lower than the actual distance. E.g. straight-line distance

    :param start: the node to start from.

    :param goal: the node we're searching for

    :return: The shortest distance to the goal node. Can be easily modified to return
the path.
    """


    # This contains the distances from the start node to all other nodes, initialized
with a distance of "Infinity"
    distances = [float("inf")] * len(graph)


    # The distance from the start node to itself is of course 0
    distances[start] = 0


    # This contains the priorities with which to visit the nodes, calculated using the
heuristic.
```

```
    priorities = [float("inf")] * len(graph)

    # start node has a priority equal to straight line distance to goal. It will be the
first to be expanded.
    priorities[start] = heuristic[start][goal]

    # This contains whether a node was already visited
    visited = [False] * len(graph)

    # While there are nodes left to visit...
    while True:
        # ... find the node with the currently lowest priority...
        lowest_priority = float("inf")
        lowest_priority_index = -1
        for i in range(len(priorities)):
            # ... by going through all nodes that haven't been visited yet
            if priorities[i] < lowest_priority and not visited[i]:
                lowest_priority = priorities[i]
                lowest_priority_index = i

        if lowest_priority_index == -1:
            # There was no node not yet visited --> Node not found
            return -1

        elif lowest_priority_index == goal:
            # Goal node found
            # print("Goal node found!")
            return distances[lowest_priority_index]

        # print("Visiting node " + lowestPriorityIndex + " with currently lowest
priority of " + lowestPriority)

        # ...then, for all neighboring nodes that haven't been visited yet....
        for i in range(len(graph[lowest_priority_index])):
            if graph[lowest_priority_index][i] != 0 and not visited[i]:
                # ...if the path over this edge is shorter...
                if distances[lowest_priority_index] + graph[lowest_priority_index][i] <
distances[i]:
                    # ...save this path as new shortest path
                    distances[i] = distances[lowest_priority_index] +
graph[lowest_priority_index][i]
```

```
                    # ...and set the priority with which we should continue with this
node
                    priorities[i] = distances[i] + heuristic[i][goal]
                    # print("Updating distance of node " + i + " to " + distances[i] +
" and priority to " + priorities[i])

                # Lastly, note that we are finished with this node.
                visited[lowest_priority_index] = True
                # print("Visited nodes: " + visited)
                # print("Currently lowest distances: " + distances)
```

Let's implement concretely this in Python.

```
graph=[['A','B',1,3],
       ['A','C',2,4],
       ['A','H',7,0],
       ['B','D',4,2],
       ['B','E',6,6],
       ['C','F',3,3],
       ['C','G',2,1],
       ['D','E',7,6],
       ['D','H',5,0],
       ['F','H',1,0],
       ['G','H',2, 0]]
temp = []
temp1 = []
for i in graph:
    temp.append(i[0])
    temp1.append(i[1])
nodes = set(temp).union(set(temp1))
def A_star(graph, costs, open, closed, cur_node):
    if cur_node in open:
        open.remove(cur_node)
    closed.add(cur_node)
    for i in graph:
        if(i[0] == cur_node and costs[i[0]]+i[2]+i[3] < costs[i[1]]):
            open.add(i[1])
            costs[i[1]] =  costs[i[0]]+i[2]+i[3]
            path[i[1]] = path[i[0]] + ' -> ' + i[1]
```

```
    costs[cur_node] = 999999
    small = min(costs, key=costs.get)
    if small not in closed:
        A_star(graph, costs, open,closed, small)
costs = dict()
temp_cost = dict()
path = dict()
for i in nodes:
    costs[i] = 999999
    path[i] = ' '
open = set()
closed = set()
start_node = input("Enter the Start Node: ")
open.add(start_node)
path[start_node] = start_node
costs[start_node] = 0
A_star(graph, costs, open, closed, start_node)
goal_node = input("Enter the Goal Node: ")
print("Path with least cost is: ",path[goal_node])
```

The output is given as

```
Enter the Start Node: A
Enter the Goal Node: H
Path with least cost is:  A -> C -> G -> H
```

The time complexity of the A* search is O(b^d) where b is the branching factor.

Advantages of A* search algorithm

- This algorithm is best when compared with other algorithms.
- This algorithm can be used to solve very complex problems also it is an optimal one.

Disadvantages of A* search algorithm

- The A* search is based on heuristics and cost. It may not produce the shortest path.
- The usage of memory is more as it keeps all the nodes in the memory.

Now, let’s compare uninformed and informed search strategies.

Comparison of uninformed and informed search algorithms

Uninformed search is also known as blind search whereas informed search is also called heuristics search. Uniformed search does not require much information. Informed search requires domain-specific details. Compared to uninformed search, informed search strategies are more efficient, and the time complexity of uninformed search strategies is more. Informed search handles the problem better than blind search.

End Notes

Search algorithms are used in games, stored databases, virtual search spaces, quantum computers, and so on. In the workbooks of the last two weeks, we have discussed some of the important search strategies and how to use them to solve the problems in AI and this is not the end. There are several algorithms to solve any problem. Nowadays, AI is growing rapidly and applies to many real-life problems. Keep learning! Keep practicing!

References

For a more detailed and comprehensive implementation of search algorithms, both uninformed and informed algorithms, please refer to:

https://github.com/aimacode/aima-python/blob/master/search.ipynb

This notebook serves as supporting material for topics covered in Chapter 3 - Solving Problems by Searching and Chapter 4 - Beyond Classical Search from the textbook of the module: “Artificial Intelligence: A Modern Approach”. This notebook uses implementations from search.py module.